MERN E-Commerce Project — Summary

Full-stack store: MongoDB, Express, React (Vite), Node.js

Current UI lives in `client/` · API in `backend/`

Document date: 15 October 2026

1. What this project is

An online shop where customers browse products, search and filter, save a wishlist, use coupons, check out with shipping, and pay with **Cash on Delivery** or **Razorpay**. Admins run a panel for products, categories, orders, users, homepage offers, and coupons.

The React app uses Vite, Tailwind, shadcn/ui, Redux Toolkit, and React Router. The API uses Express, Mongoose, JWT, Cloudinary for images, and Razorpay for cards/UPI.

2. What people can do

Shoppers (guest)

- Home: offer banners, categories, featured and latest products.
- Shop: search, category filter, sort, pagination; product cards with wishlist and add-to-cart.
- Product page: gallery, price, stock, quantity, reviews, add to cart / wishlist.
- Cart: quantities, remove items, apply coupon, proceed to checkout (login required).

Logged-in customers

- Register, login, forgot/reset password; edit profile (name, email, password).
- Save a shipping address; wishlist; view order history and a single order.
- Checkout: shipping → payment method (COD or Razorpay) → review/place order → pay if online.

Admins

- Dashboard counts (products, orders, users).
- CRUD products (image upload to Cloudinary), categories, offers/banners, coupons.
- List users and delete them; list orders, open details, mark delivered.

3. All pages and routes

Public store

Route	Page	What it does
`/`	Home	Banners, categories, featured/latest products, shop CTA.
`/shop`	Shop	Catalog with search, filters, sort, pagination.
`/product/:id`	Product details	Full product, reviews, cart and wishlist actions.
`/cart`	Cart	Line items, coupon, totals, checkout.

Auth (logged-out users only)

Route	Page	What it does
`/login`	Login	Email/password sign-in.
`/register`	Register	Create account.
`/forgot-password`	Forgot password	Request a reset.
`/reset-password`	Reset password	Set a new password from the reset link.

Account & checkout (must be logged in)

Route	Page	What it does
`/account`	Overview	Welcome, shortcuts to profile, orders, wishlist.
`/account/profile`	Profile	Update name, email, password.
`/account/address`	Address	Save, edit, or remove shipping address.
`/account/wishlist`	Wishlist	Saved products; add to cart.
`/shipping`	Shipping	Checkout step 1: delivery address.
`/payment`	Payment	Checkout step 2: COD or Razorpay.
`/placeorder`	Place order	Checkout step 3: review, coupon, create order, pay.
`/orders`	My orders	List of the user's orders.
`/order/:id`	Order details	Items, address, paid/delivered, pay if unpaid.

Admin panel (admin users only)

Route	Page	What it does
`/admin`	Dashboard	Counts and recent activity snapshot.
`/admin/products`	Products	Search catalog; edit or delete.
`/admin/products/new`	Create product	Name, price, brand, category, stock, image, description.
`/admin/products/:id/edit`	Edit product	Update the same fields.
`/admin/orders`	Orders	All store orders; open one.
`/admin/orders/:id`	Order details	Customer, items, payment; mark delivered.
`/admin/users`	Users	List accounts; delete a user.
`/admin/categories`	Categories	Create, edit, delete shop categories.
`/admin/offers`	Offers	Homepage banners; toggle active; edit/delete.
`/admin/offers/new`	Create offer	Title, text, image, button, dates, active flag.
`/admin/offers/:id/edit`	Edit offer	Update a banner.
`/admin/coupons`	Coupons	Discount codes; edit/delete.
`/admin/coupons/new`	Create coupon	Code, type, value, limits, dates.

`/admin/coupons/:id/edit`	Edit coupon	Update a coupon.

4. Checkout flow

Cart → Login (if needed) → Shipping → Payment method → Place order → Razorpay (if online) or wait for COD → Order details. Cart is in Redux/localStorage; coupons are checked against the API.

5. Project structure

```
mern-ecommerce-project-main/
├── backend/                    Express API
│   ├── server.js               Routes, CORS, JSON, static /uploads
│   ├── config/                 db, cloudinary, razorpay
│   ├── models/                 User, Product, Order, Category, Offer, Coupon
│   ├── controllers/            Business logic
│   ├── routes/                 /api/products, users, orders, upload,
│   │                           categories, offers, coupons, payments, health
│   ├── middleware/             JWT protect/admin, multer
│   └── utils/generateToken.js
└── client/                     React storefront + admin
    └── src/
        ├── routes/             AppRoutes, Protected, Admin, Auth
        ├── layouts/            Store, Account, Admin
        ├── pages/              public, auth, account, cart, checkout,
        │                       orders, admin
        ├── components/         layout, product, cart, admin forms, ui
        ├── store/              auth, cart, wishlist
        └── services/           Axios API helpers
```

6. Backend API (brief)

Area	Main endpoints
Health	GET /api/health
Users	POST register/login; GET/PUT profile; GET users & DELETE user (admin)
Products	List/search, featured, by id, reviews; admin create/update/delete
Orders	Create, my orders, by id, pay; admin list + mark delivered
Categories / offers / coupons	Public read; admin write
Upload	POST /api/upload (images → Cloudinary)
Payments	POST /api/payments/razorpay/create-order and /verify

7. How to run

1. Backend `.env`: `MONGO_URI`, `JWT_SECRET`, `PORT` (optional, default 5000), `FRONTEND_URL` (e.g. http://localhost:5173), Cloudinary keys, Razorpay `KEY_ID` / `KEY_SECRET`.
2. `cd backend` → `npm install` → `npm run server`.
3. Client: `VITE_API_URL` pointing at the API (e.g. http://localhost:5000).
4. `cd client` → `npm install` → `npm run dev`.

> Admin screens are under `/admin/*` and need a user with `isAdmin: true`. Do not seed or wipe a production database.

8. Tech stack

Client: React 19, Vite 8, React Router 7, Redux Toolkit, Axios, Tailwind 4, shadcn/ui, react-hook-form, Zod, Razorpay checkout.
Backend: Express, Mongoose, JWT, bcryptjs, Multer, Cloudinary, Razorpay, CORS, dotenv.

End of summary — based on current `client` routes and backend API.